

BUILD ROADMAP · ENGINEERING & STRATEGY

Wathiq

وثيق

A runtime governance & tamper-evident provenance control plane for AI workflows — built for UAE & MENA regulated industries.

Prepared for: Founder / Lead Engineer · **Horizon:** 12-week solo build (full-time)

Wedge: Tamper-evident provenance first → runtime enforcement for CBUAE-regulated financial institutions

Status: Working document — revise as you learn. Treat every date as a target, not a contract.

How to read this document

This roadmap assumes you can write basic code but have never shipped production infrastructure. It explains *what* to do each day, *why* it matters, and gives you real code to start from. You will not understand every line on day one — that is normal. Build, run, break, fix, repeat.

The document is organised into **four phases over twelve weeks**. Each phase ends in something you can show another human. The colour-coded boxes mean:

WHY THIS MATTERS

The reasoning behind a decision. Read these even when skimming — they are how you learn to think like an architect, not just a coder.

REALITY CHECK

A constraint, risk, or place beginners get burned. Ignoring these costs days.

DONE WHEN

The concrete test that proves a task is finished. If you can't tick it, you're not done.

REGULATORY NOTE

How a step maps to UAE PDPL or the CBUAE AI Guidance Note. This is your sales story — keep it close.

Table of contents

Part 0 — Mental model: what you are actually building	p.3
Phase 1 (Weeks 1–4) — Tamper-evident provenance MVP	p.5
Week 1 — Foundations, environment, first signed event	
Week 2 — Hash chains, the database, the SDK	
Week 3 — Merkle trees & external anchoring (the unfakeable part)	
Week 4 — Replay, verification & evidence export	
Phase 2 (Weeks 5–7) — Multi-tenancy, PII & the dashboard	p.16
Phase 3 (Weeks 8–11) — The enforcement layer & policy engine	p.22

Phase 4 (Week 12) — Pilot hardening, evidence bundle & the pitch	p.29
Appendix A — The reference policy pack (CBUAE + PDPL mapped)	p.31
Appendix B — Glossary for the newbie engineer	p.34
Appendix C — Honest limitations & what NOT to claim	p.35

Part 0 — The mental model

Before any code, hold this picture in your head. Everything you build serves one sentence you will say to a regulator:

THE ONE SENTENCE

“You do not have to trust us, or our customer. Here is independent cryptographic proof that this exact AI action happened at this exact time, under these exact policies, and has not been altered since — and where required, proof that its content was provably destroyed.” If a feature does not serve that sentence, it is a distraction for now.

Wathiq does two jobs

Job 1 — Provenance (record the truth). Sit beside an AI workflow and capture every step — the prompt, the model output, the tool calls, the policy that applied — as a signed, chained, externally-anchored record that can be replayed and exported. This is *passive*: it never blocks anything. This is what you build first (Phase 1).

Job 2 — Enforcement (gate the risk). Before a risky AI action executes, evaluate the institution’s policies, score the risk, and decide: allow, flag, auto-mitigate, or require a human approval. This is *active*: it sits in the critical path. This is the deep-tech, higher-value layer you build last (Phase 3).

REALITY CHECK

These are almost two different products with different failure rules. Provenance can be slow and asynchronous; nobody waits on it. Enforcement must be fast and its downtime can stop your customer’s production AI. Never let enforcement logic creep into Phase 1 — it will balloon your scope and you will ship nothing.

The architecture in one diagram (in words)

A customer’s AI app talks to the **Wathiq SDK** (a small Python library they install). The SDK does two things: for the common, low-risk case it decides locally in microseconds and *buffers an event*; for the rare risky case it calls the **Wathiq API** (a FastAPI server you run). The API writes events to **PostgreSQL**, signs them, batches them into a **Merkle tree**, and **anchors** the batch’s root hash to an external authority (a timestamping service, and later a public log). A **Next.js dashboard** lets humans see pending approvals and replay/verify any trace. Signing keys live in a **cloud KMS** in the UAE region so neither you nor the customer can secretly forge records.

Component	What it is	Phase
Wathiq API	FastAPI (Python) server: ingest events, sign, anchor, replay, export	1
PostgreSQL	The database storing orgs, traces, events, approvals	1
Wathiq SDK	Python library customers install to record events & (later) precheck	1–3
Signing + Merkle + Anchor	The cryptographic core — your real moat	1
Multi-tenancy + PII pipeline	Org isolation, redaction, crypto-shredding for PDPL erasure	2
Dashboard	Next.js UI: approvals queue, trace viewer, evidence export	2–4
Policy engine + risk model	The enforcement brain: rules, detection layers, tiers	3

REGULATORY NOTE

The UAE has deliberately **not** written a standalone AI law — AI must comply with existing laws (PDPL, the CBUAE AI Guidance Note of Feb 2026, sectoral rules). That is good for you: institutions need tooling to satisfy expectations that have no prescribed product. The CBUAE Guidance requires a documented AI governance framework, a model inventory, human oversight of automated outcomes, and retained documentation of risk assessments and controls. Provenance answers the inventory + documentation requirement; enforcement answers the human-oversight requirement. You are selling the two controls the regulator already named.

The non-negotiable rule for the next 12 weeks

DONE WHEN (THE WHOLE PROJECT)

You can sit across from a financial-institution risk officer, run a live AI workflow through Wathiq, then hand them an exported evidence bundle and a verification script they run themselves that prints **ALL EVENTS VERIFIED** — without trusting your word. Everything below builds toward that single demo.

Phase 1 (Weeks 1–4) — Tamper-evident provenance MVP

Goal of this phase: a working API + SDK that records AI events as a signed, chained, externally-anchored log you can replay and verify. By the end you have the unfakeable core that even Hashirai treats as “optional anchoring” — and you make it the default.

WEEK 1 Foundations, environment, and your first signed event

Day 1 — Set up your machine and understand the tools

Install the toolchain. On macOS use Homebrew; on Windows use WSL2 (a Linux environment) so the commands match. You need: **Python 3.11+**, **Docker** (to run PostgreSQL without installing it directly), **Git**, and a code editor (VS Code).

```
# Check you have the basics (each should print a version)
python3 --version
git --version
docker --version

# Create the project and a Python virtual environment
# A "venv" is an isolated box of libraries just for this project,
# so you never pollute your system Python.
mkdir wathiq && cd wathiq
python3 -m venv .venv
source .venv/bin/activate          # on Windows WSL: same; on
PowerShell: .venv\Scripts\activate
python -m pip install --upgrade pip
```

WHY THIS MATTERS

A virtual environment means “it works on my machine” problems mostly disappear, because the exact library versions live in `.venv` and are listed in a file. Professional teams never install project libraries globally.

Day 2 — Initialise Git and the project skeleton

```
# Start version control. Commit early and often – it's your undo button.
git init
printf ".venv/\n__pycache__/\n*.pyc\n.env\nkeys/\n" > .gitignore

# Folder layout we will grow into:
mkdir -p app sdk tests scripts keys
touch app/__init__.py app/main.py
echo "# Wathiq" > README.md
git add . && git commit -m "chore: project skeleton"
```

REALITY CHECK

The `.gitignore` line for `keys/` and `.env` is critical. You must **never** commit private signing keys or secrets to Git. Leaked keys = your entire tamper-evidence guarantee is void. Treat this as a hard safety rule, not a style preference.

Day 3 — Install FastAPI and run a “hello” server

FastAPI is the web framework for your API. It is fast, modern, and gives you free interactive docs.

```
pip install "fastapi[standard]" uvicorn

# Save versions so the build is reproducible
pip freeze > requirements.txt
```

Put this in `app/main.py`:

```
from fastapi import FastAPI

app = FastAPI(title="Wathiq API", version="0.1.0")

@app.get("/health")
def health():
    # A health check: lets you (and later, monitoring) confirm the server is alive.
    return {"status": "ok", "service": "wathiq"}
```

```
# Run it (the server reloads automatically when you edit code):
uvicorn app.main:app --reload

# Visit http://127.0.0.1:8000/health -> you should see the JSON.
# Visit http://127.0.0.1:8000/docs -> interactive API explorer. FastAPI gives this free.
```

DONE WHEN

Browsing to `/health` returns `{"status":"ok"...}` and `/docs` shows a clickable interface.

Day 4 — Learn the single most important concept: canonicalization

To prove data hasn't changed, you compute a **hash** of it (a short fingerprint). But the same data can be written many ways — `{"a":1, "b":2}` and `{"b":2, "a":1}` mean the same thing but produce *different* fingerprints. So before hashing you must convert data into **one canonical form**: keys sorted, no random spacing, fixed encoding. Same meaning → same bytes → same hash, every time.

```
import json, hashlib

def canonical_bytes(obj: dict) -> bytes:
    # sort_keys=True -> deterministic ordering
    # separators -> no incidental whitespace
    # ensure_ascii=False + utf-8 -> stable handling of Arabic/Unicode text
    return json.dumps(
        obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False
    ).encode("utf-8")

def sha256_hex(obj: dict) -> str:
    return hashlib.sha256(canonical_bytes(obj)).hexdigest()

print(sha256_hex({"b": 2, "a": 1}))
print(sha256_hex({"a": 1, "b": 2})) # identical to the line above – proof it works
```

WHY THIS MATTERS

Canonicalization is the foundation everything else rests on. If your verifier ever rebuilds the object even slightly differently from how you hashed it (different key order, a stray space, a date formatted two ways), verification will fail on honest data and you'll waste days. Pick the rules once, write them down, and use the *exact same function* on the write path and the verify path.

Day 5 — Generate signing keys and sign your first hash

A hash proves data is unchanged. A **signature** proves *who* vouched for it. You use **Ed25519**, a modern signature scheme: a private key signs, a public key verifies. You keep the private key secret; anyone can verify with the public key.

```
pip install cryptography
```



```

from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey, Ed25519PublicKey)
from cryptography.hazmat.primitives import serialization
import pathlib

KEYS = pathlib.Path("keys"); KEYS.mkdir(exist_ok=True)

def generate_keys():
    priv = Ed25519PrivateKey.generate()
    pub = priv.public_key()
    (KEYS / "ed25519_private.pem").write_bytes(
        priv.private_bytes(
            serialization.Encoding.PEM,
            serialization.PrivateFormat.PKCS8,
            serialization.NoEncryption())) # dev only; production uses KMS
    (KEYS / "ed25519_public.pem").write_bytes(
        pub.public_bytes(
            serialization.Encoding.PEM,
            serialization.PublicFormat.SubjectPublicKeyInfo))
    print("Keys written to ./keys")

if __name__ == "__main__":
    generate_keys()

```

```

# Sign a hash and verify it:
def sign_hex(private_pem: bytes, message_hex: str) -> str:
    priv = serialization.load_pem_private_key(private_pem, password=None)
    sig = priv.sign(bytes.fromhex(message_hex))
    return sig.hex()

def verify_hex(public_pem: bytes, message_hex: str, signature_hex: str) -> bool:
    pub = serialization.load_pem_public_key(public_pem)
    try:
        pub.verify(bytes.fromhex(signature_hex), bytes.fromhex(message_hex))
        return True
    except Exception:
        return False

```

REALITY CHECK

Right now the private key sits in a file on disk — fine for development, **fatal for production**. Anyone who reads that file can forge any record. In Phase 4 you migrate signing to a cloud KMS/HSM in the UAE region, where the key never leaves the hardware and every signing call is itself logged. Write a **TODO: move to KMS** comment now so you don't forget.

DONE WHEN

You can run a script that generates keys, signs the canonical hash of a sample event, and verifies it returns **True** — and returns **False** if you change one character of the data.

Day 6–7 — Stand up PostgreSQL and design the schema

Run PostgreSQL in Docker so you don't install it system-wide:

```
docker run --name wathiq-db -e POSTGRES_PASSWORD=devpass \
  -e POSTGRES_DB=wathiq -p 5432:5432 -d postgres:16

pip install sqlalchemy psycopg2-binary alembic pydantic-settings
```

Your core tables (simplified for the MVP). A **trace** is one AI workflow run; an **event** is one step in it. Each event stores the hash of the previous event (`prev_hash`) — that is the “chain” that makes tampering detectable.

```
-- events: the heart of the system
CREATE TABLE events (
  id          UUID PRIMARY KEY,
  org_id      TEXT NOT NULL,
  trace_id    UUID NOT NULL,
  seq         INTEGER NOT NULL,          -- 1,2,3... order within a trace
  type        TEXT NOT NULL,            -- model_completion | tool_call | policy_decision |
  approval
  payload_hash TEXT NOT NULL,           -- hash of the (separately stored) content
  envelope     JSONB NOT NULL,          -- the permanent structural metadata
  prev_hash    TEXT,                    -- hash of previous event in this trace
  hash         TEXT NOT NULL,           -- hash of THIS event's canonical envelope
  signature    TEXT NOT NULL,           -- Ed25519 signature over hash
  policy_version TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (trace_id, seq)                -- stops duplicate/out-of-order seq
);
```

WHY THIS MATTERS

Notice `envelope` (permanent metadata) is separate from the content, and we only store `payload_hash` here. The actual prompt/output lives in a *separate* encrypted store (Phase 2). This split is what lets you honor a PDPL erasure request — destroy the content, keep the proof-of-existence — without breaking the chain. Designing it in on day one saves a painful rewrite later.

Day 8 — The append-only event service (chaining)

When a new event arrives: look up the last event for that trace, take its hash as your `prev_hash`, build the canonical envelope, hash it, sign it, store it.

```
def build_envelope(trace_id, seq, etype, payload_hash, prev_hash, policy_version, created_at):
    # This dict is what gets hashed. Keep its shape IDENTICAL on write and verify.
    return {
        "trace_id": str(trace_id),
        "seq": seq,
        "type": etype,
        "payload_hash": payload_hash,
        "prev_hash": prev_hash,
        "policy_version": policy_version,
        "created_at": created_at,  # ISO-8601 string, server UTC
    }

def record_event(db, org_id, trace_id, seq, etype, payload: dict, policy_version):
    last = db.last_event(trace_id)  # None if first event
    prev_hash = last.hash if last else None
    if last and seq != last.seq + 1:
        raise ValueError(f"out-of-order seq: expected {last.seq+1}, got {seq}")
    created_at = now_utc_iso()
    payload_hash = sha256_hex(payload)
    envelope = build_envelope(trace_id, seq, etype, payload_hash,
                              prev_hash, policy_version, created_at)
    h = sha256_hex(envelope)
    sig = sign_hex(load_private_pem(), h)
    db.insert_event(org_id, trace_id, seq, etype, payload_hash,
                   envelope, prev_hash, h, sig, policy_version, created_at)
    db.store_payload(payload_hash, payload)  # separate content store
    return {"hash": h, "signature": sig, "seq": seq}
```

REALITY CHECK

The server sets `created_at` using its own UTC clock, never the client's. Client clocks lie, drift, and can be spoofed. Treat any client-supplied timestamp as untrusted metadata only. This is a classic beginner mistake that destroys the credibility of a time-ordered log.

Day 9 — Expose it as the `/v1/event` endpoint

```
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

class EventIn(BaseModel):
    trace_id: str
    seq: int
    type: str
    payload: dict
    policy_version: str | None = None

@app.post("/v1/event")
def post_event(body: EventIn, x_api_key: str = Header(...)):
    org = resolve_org(x_api_key)  # auth: map API key -> org
    if not org:
        raise HTTPException(401, "invalid api key")
    return record_event(db, org.id, body.trace_id, body.seq,
                       body.type, body.payload, body.policy_version)
```

DONE WHEN

Using `/docs` you can POST three events to one `trace_id` with seq 1,2,3 and see each returns a hash, and the 2nd/3rd include the previous hash. Posting seq 5 next is rejected.

Day 10 — Build the Python SDK (the thing customers actually touch)

The SDK is a thin, friendly wrapper so a customer writes 2 lines, not 20. Keep it tiny.

```
# sdk/wathiq.py
import requests, uuid

class WathiqClient:
    def __init__(self, api_key, base_url="http://localhost:8000"):
        self.api_key = api_key
        self.base_url = base_url.rstrip("/")

    def new_trace(self) -> str:
        return str(uuid.uuid4())

    def record_event(self, trace_id, seq, type, payload, policy_version=None):
        r = requests.post(
            f"{self.base_url}/v1/event",
            headers={"x-api-key": self.api_key},
            json={"trace_id": trace_id, "seq": seq, "type": type,
                "payload": payload, "policy_version": policy_version},
            timeout=5,
        )
        r.raise_for_status()
        return r.json()
```

```
# How a customer (e.g. Tradeeasy) would use it:
w = WathiqClient(api_key="org_demo_key")
t = w.new_trace()
w.record_event(t, 1, "model_completion",
    {"prompt": "Summarise Q1 market", "output": "...", "citations": 3})
```

WHY THIS MATTERS

Integration friction is the #1 reason governance tools die in pilots. If a customer must change 50 lines to adopt you, they won't. Two lines — “make a trace, record an event” — is the bar. Later the SDK also buffers events and does local prechecks, but it must always stay this simple on the surface.

WEEK 3 Merkle trees & external anchoring — the part nobody can fake

WHY THIS MATTERS

A signed hash chain only stops a *lazy* attacker. Because **you** hold the signing key and run the server, you (or your customer) could rewrite the whole chain and re-sign it. To make the record unforgeable *even against yourself* — which is your stated threat model and your core differentiator over Hashirai — you must commit the record to a place you cannot later edit. That is anchoring. This week is the most important in the whole project.

Day 11–12 — Build a Merkle tree

A Merkle tree takes many event hashes and combines them, pair by pair, into a single **root hash**. Change any one event and the root changes. It lets you later prove “event X was in this batch” with a tiny proof, without revealing the other events.

```
import hashlib

def _h(b: bytes) -> bytes:
    return hashlib.sha256(b).digest()

def merkle_root(leaf_hexes: list[str]) -> str:
    if not leaf_hexes:
        return _h(b"").hex()
    level = [bytes.fromhex(x) for x in leaf_hexes]
    while len(level) > 1:
        if len(level) % 2 == 1:          # odd count: duplicate the last leaf
            level.append(level[-1])
        level = [_h(level[i] + level[i+1]) for i in range(0, len(level), 2)]
    return level[0].hex()
```

Day 13 — Batch events and compute the root on a schedule

Every N minutes, gather all events recorded since the last batch, compute their Merkle root, and store a **batch record** (root + time range + which events). Sign the root with your KMS key (file key for now).

```
def seal_batch(db):
    events = db.unbatched_events()          # since last batch
    if not events:
        return None
    root = merkle_root([e.hash for e in events])
    sig = sign_hex(load_private_pem(), root)
    batch_id = db.insert_batch(root=root, signature=sig,
                               event_ids=[e.id for e in events],
                               sealed_at=now_utc_iso())
    return {"batch_id": batch_id, "root": root, "signature": sig}
```

Day 14 — Anchor the root externally (RFC 3161 timestamping)

Send the root hash to a free, standardised **Timestamping Authority (TSA)**. It returns a signed token proving “I saw this hash at this time.” Now you cannot backdate or alter anything before that timestamp, because you don’t hold the TSA’s key.

```
pip install rfc3161-client # or call a TSA with openssl; concept is identical
```

```
# Conceptual flow (library APIs vary – read the docs for the exact calls):
# 1. Build a timestamp request over the Merkle root bytes.
# 2. POST it to a public TSA (e.g. freeTSA, DigiCert) over HTTP.
# 3. Store the returned timestamp token next to the batch row.
def anchor_batch(db, batch_id, root_hex):
    token = request_tsa_timestamp(bytes.fromhex(root_hex)) # returns bytes
    db.save_anchor(batch_id, kind="rfc3161", token=token, anchored_at=now_utc_iso())
```

REALITY CHECK

For the MVP, RFC 3161 timestamping is enough and is well understood by auditors. A public-blockchain or Certificate-Transparency-style log (e.g. Google’s open-source Trillian) is *stronger* but more work — schedule it as a Phase-4 “nice to have,” not an MVP blocker. Do not let blockchain hype pull you into building a chain integration before you have a working timestamp anchor.

REGULATORY NOTE

External anchoring is what turns “our logs” (which a regulator distrusts because you control them) into “independently verifiable evidence.” When you later export an evidence bundle, the TSA token is the line that lets the regulator verify the timeline *without trusting you*. This single capability is your strongest answer to “why not just use AWS CloudTrail / provider logs?”

WEEK 4 Replay, verification & the evidence export

Day 15–16 — The replay/verify endpoint

Given a `trace_id`, load its events in order and independently re-verify each one: rebuild the canonical envelope, recompute the hash, check it matches the stored hash, check the signature, and check the chain links (each `prev_hash` equals the previous event’s `hash`).

```

@app.get("/v1/trace/{trace_id}/replay")
def replay(trace_id: str, x_api_key: str = Header(...)):
    org = resolve_org(x_api_key) or _401()
    events = db.events_for_trace(org.id, trace_id) # ordered by seq
    results, prev_hash, all_ok = [], None, True
    pub = load_public_pem()
    for e in events:
        rebuilt = build_envelope(e.trace_id, e.seq, e.type, e.payload_hash,
                                e.prev_hash, e.policy_version, e.created_at)
        recomputed = sha256_hex(rebuilt)
        hash_ok = (recomputed == e.hash)
        sig_ok = verify_hex(pub, e.hash, e.signature)
        chain_ok = (e.prev_hash == prev_hash)
        ok = hash_ok and sig_ok and chain_ok
        all_ok = all_ok and ok
        results.append({"seq": e.seq, "type": e.type, "verified": ok,
                      "hash_ok": hash_ok, "signature_ok": sig_ok, "chain_ok": chain_ok})
        prev_hash = e.hash
    return {"trace_id": trace_id, "all_verified": all_ok, "events": results}

```

DONE WHEN

Replay returns `all_verified: true` for an untouched trace. Then manually edit one event's stored data in the DB and replay again — it must return `false` and point at the exact failing event.

Demonstrating this break-and-detect live is your single most persuasive demo moment.

Day 17–18 — The evidence bundle export

Export everything a third party needs to verify offline, with no access to your servers: the events, the batch root + Merkle proofs, the TSA anchor token, the public key, and a plain-English README of verification steps.

```

@app.get("/v1/evidence/{trace_id}/export")
def export_evidence(trace_id: str, x_api_key: str = Header(...)):
    org = resolve_org(x_api_key) or _401()
    events = db.events_for_trace(org.id, trace_id)
    batch = db.batch_for_events([e.id for e in events])
    bundle = {
        "trace_id": trace_id,
        "exported_at": now_utc_iso(),
        "public_key_pem": load_public_pem().decode(),
        "events": [serialize_event(e) for e in events],
        "batch": {"root": batch.root, "signature": batch.signature,
                  "merkle_proofs": build_proofs(events, batch)},
        "anchor": db.anchor_for_batch(batch.id), # RFC3161 token (base64)
        "verification_instructions": VERIFY_README,
    }
    return bundle # also offer as downloadable .zip

```

WHY THIS MATTERS

Ship a tiny standalone `verify.py` inside every bundle that re-checks all of the above using only the bundle's contents and public libraries. When a regulator runs *your* script and sees **ALL EVENTS VERIFIED** on their own laptop, you have moved from “trust us” to “verify us.” That is the entire product thesis made tangible.

Day 19–20 — Instrument one real workflow + write tests

Wire the SDK into one Tradeeasy-style workflow (generate a market summary → record prompt, output, citations as events). Then write automated tests so future changes can't silently break verification.

```
pip install pytest

# tests/test_chain.py – the test that protects your core guarantee
def test_tampering_is_detected(client):
    t = make_trace_with_three_events(client)
    assert client.replay(t)["all_verified"] is True
    corrupt_one_event_in_db(t, seq=2)
    res = client.replay(t)
    assert res["all_verified"] is False
    assert res["events"][1]["verified"] is False      # the exact tampered event
```

PHASE 1 DONE WHEN

(1) SDK records events from a real workflow; (2) replay verifies an intact trace and detects tampering; (3) a batch is sealed, Merkle-rooted, and anchored to a TSA; (4) an evidence bundle exports and its bundled `verify.py` passes on a clean machine; (5) `pytest` is green. *You now have a demoable provenance product — the same category Hashirai sells, but with anchoring on by default and a clear path to UAE-native enforcement they don't have.*

Phase 2 (Weeks 5–7) — Multi-tenancy, PII & the dashboard

Phase 1 proved the core. Now make it safe to put real customer data in, and give humans a window into it. This is where PDPL compliance becomes concrete.

WEEK 5 Multi-tenancy & API-key security

Day 21–22 — Org isolation

“Multi-tenant” means many customer organisations share one system but can never see each other’s data. Every table already has `org_id`; the rule is that **every query is filtered by the `org_id` of the authenticated caller** — no exceptions.

```
-- orgs table
CREATE TABLE orgs (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  api_key_hash TEXT NOT NULL,      -- store a HASH, never the raw key
  region      TEXT DEFAULT 'uae',
  retention_days_payload INTEGER DEFAULT 365,
  fail_mode   TEXT DEFAULT 'deny_on_error', -- or 'allow_with_flag'
  created_at  TIMESTAMPTZ DEFAULT now()
);
```

REALITY CHECK

Store only a **hash** of each API key (e.g. SHA-256), never the key itself. If your DB leaks, hashed keys can’t be used to impersonate customers. When a request arrives, hash the incoming key and look up the match. This is the same reason websites never store your raw password.

WHY THIS MATTERS

A single missing `WHERE org_id = ?` is a cross-tenant data leak — the kind of bug that ends a governance company instantly, because your entire pitch is trustworthiness. Centralise data access behind functions that *require* an `org_id` argument, so it’s impossible to forget.

Day 23 — Per-org fail-mode & region

Record each org’s `fail_mode`: regulated/government customers want **fail-closed** (`deny_on_error`) — if Wathiq can’t evaluate, block the risky action. Others want **fail-open** (`allow_with_flag`) so your downtime never halts their product. `region` drives where data and keys live.

Day 24–25 — Detect and redact PII before storage

Before any content is stored, scan it for personal data (Emirates ID, IBAN, phone, email, names) and redact or tokenise it. Start with high-precision regex for structured IDs; add an ML/LLM pass for names later.

```
import re
PATTERNS = {
    "emirates_id": re.compile(r"\b784-?\d{4}-?\d{7}-?\d\b"),
    "iban_ae": re.compile(r"\bAE\d{2}\d{3}\d{16}\b"),
    "email": re.compile(r"\b[\w.+-]+@[ \w-]+\.[\w.-]+\b"),
    "phone_ae": re.compile(r"\b(?:\+?971|0)5\d{8}\b"),
}
def redact(text: str) -> tuple[str, list[str]]:
    found = []
    for label, rx in PATTERNS.items():
        if rx.search(text):
            found.append(label)
            text = rx.sub(f"[REDACTED:{label}]", text)
    return text, found
```

REGULATORY NOTE

PDPL requires *data minimisation* and *privacy by design*, and the 2026 guidance expects documented DPIAs and audit-ready handling. Redacting PII before it ever lands in your store, and recording *that you did so* as an event, is a concrete, demonstrable PDPL control — not a checkbox claim.

Day 26–27 — Crypto-shredding for the “right to erasure”

PDPL gives individuals a right to have their data erased — but your provenance says nothing changes. You resolve this by storing content **encrypted with a per-record key**, separate from the permanent envelope. To “erase,” you destroy the key: the content becomes unrecoverable noise, while the hash-proof of existence survives.

```
from cryptography.fernet import Fernet
def store_payload_encrypted(payload_hash, plaintext: bytes):
    key = Fernet.generate_key() # unique per record
    blob = Fernet(key).encrypt(plaintext)
    content_store.put(payload_hash, blob)
    key_store.put(payload_hash, key) # the deletable part

def erase_payload(payload_hash, approver_id, reason):
    key_store.delete(payload_hash) # crypto-shred: content now unrecoverable
    record_event(... type="erasure", # the deletion is itself an anchored event
                 payload={"target": payload_hash, "approver": approver_id, "reason": reason})
```

REALITY CHECK

Erasure is a tampering risk in disguise — a bad actor could “erase” only the incriminating event.

Defences: (1) erasure requires the same human-approval flow as a high-risk action; (2) the erasure is itself a recorded, anchored event so the deletion can never be hidden; (3) you erase *content* but never an event's *existence/metadata*. The trace then shows “content here was destroyed, by whom, when, why” — provably, not silently.

DONE WHEN

You can erase a payload, the content is unreadable afterward, the trace still verifies structurally, and the replay shows an explicit, signed erasure event in the timeline.

WEEK 7 The dashboard (Next.js)

Day 28–30 — Trace viewer + verification status

Build a minimal Next.js app with three screens: a list of traces, a trace detail view that calls `/replay` and shows a green/red verified badge per event, and an evidence-export button. Keep it plain — this is a tool for risk officers, not a consumer app.

```
npx create-next-app@latest wathiq-dashboard --typescript --app
cd wathiq-dashboard && npm install
```

```
// Fetching a trace's verification result (App Router, server component)
async function getTrace(id: string) {
  const res = await fetch(`${process.env.API_URL}/v1/trace/${id}/replay`, {
    headers: { "x-api-key": process.env.WATHIQ_KEY! }, cache: "no-store",
  });
  return res.json();
}
```

Day 31–35 — Pending approvals queue (scaffold for Phase 3)

Add a screen that lists approvals with `status = 'pending'` and Approve/Deny buttons that POST to an approvals endpoint. You won't fully wire enforcement until Phase 3, but building the human-review UI now means the enforcement layer has somewhere to send Red-tier cases.

PHASE 2 DONE WHEN

Real (redacted) data flows safely; orgs are isolated; you can crypto-shred on request with an anchored erasure event; and a dashboard shows live verification status and an approvals queue.

Phase 3 (Weeks 8–11) — The enforcement layer & policy engine

The deep-tech, higher-value layer, and the one Hashirai does not do. Build it on the provenance foundation so every enforcement decision is itself a signed, anchored event.

REALITY CHECK — READ BEFORE CODING

You cannot, solo, in four weeks, build a system that “knows the law” and autonomously decides legality. Do not try, and never market it that way — false confidence in a regulated workflow is exactly what gets someone sued. Reframe: **Wathiq enforces the institution’s own documented policies and proves it did.** The institution owns the policy (the CBUAE Guidance puts accountability on *their* board, not on tooling); you are the control-and-evidence layer with sensible MENA-aware starter templates they customise.

WEEK 8 The precheck endpoint & risk tiers

Day 36–37 — The decision contract

Before a risky action runs, the customer calls `precheck`. It returns a decision, a risk tier, a score, and crucially a human-readable **reason**. Four tiers:

Tier	Meaning	Default action
GREEN	Safe	Allow, record event
YELLOW	Minor concern	Allow + flag for later review
ORANGE	Moderate	Allow + notify / apply auto-mitigation
RED	High risk	Block & require human approval

```
@app.post("/v1/precheck")
def precheck(body: PrecheckIn, x_api_key: str = Header(...)):
    org = resolve_org(x_api_key) or _401()
    try:
        features = extract_features(body.action_type, body.meta, body.content)
        decision = evaluate_policies(org.id, body.action_type, features)
    except Exception:
        decision = fail_decision(org.fail_mode) # fail-closed or fail-open
    record_event(... type="policy_decision", payload=decision) # every decision is evidence
    if decision["tier"] == "red":
        decision["approval_id"] = open_approval(org.id, body.trace_id, decision)
    return decision
```

Day 38–40 — Layered detection (cheapest first)

Quality comes from layering detectors, each emitting a confidence and a reason — never a naked yes/no.

1. **Deterministic** (regex/structural): PII present? citation count == 0? prohibited phrase? Runs in microseconds, catches the boring majority.
2. **Classifier** (ML/LLM-as-judge): “does this output give individualised financial advice?” via a tightly-constrained rubric prompt returning structured JSON + confidence. Used only when layer 1 is ambiguous or the action is medium-risk. Slower; budget for it.
3. **Policy evaluation**: combine features + metadata against the org’s rules → tier + decision + reason.

WHY THIS MATTERS

RED always means *human approval*, never autonomous block-and-forget. The CBUAE Guidance *requires* that customers can get human intervention on automated outcomes — so your detector’s imperfection is not a weakness, it is the compliant design. You turn “our classifier isn’t perfect” into “we surface risk for human judgment with a full evidentiary record.”

WEEK 9 The policy engine

Day 41–43 — JSON rules now, OPA/Rego-ready later

Start with simple JSON rules (see Appendix A). Design the engine’s input/output contract so you can later swap the internals for OPA/Rego (an industry-standard policy language) *without changing the SDK*.

```
def evaluate_policies(org_id, action_type, features):
    rules = load_active_rules(org_id) # sorted by priority desc
    for rule in rules:
        if matches(rule["match"], action_type, features) and \
            condition_holds(rule["condition"], features):
            return {"decision": rule["decision"], "tier": rule["tier"],
                    "rule": rule["id"], "reason": rule["reason"],
                    "risk_score": score(features), "policy_version": rules.version}
    return {"decision": "allow", "tier": "green", "rule": None,
            "reason": "no rule matched", "risk_score": score(features)}
```

Day 44–45 — Auto-mitigation (Yellow/Orange)

For mild issues, return mitigation instructions the SDK can apply client-side: redact PII, soften absolute claims, append a “verify with primary source” disclaimer. Record the mitigation as an event.

WEEK 10 Approvals flow end-to-end

Day 46–48 — Wire Red → dashboard → resume

Red precheck opens a pending approval → appears in the dashboard queue → a reviewer approves/denies with a comment → an `approval_action` event is recorded and anchored → the customer’s workflow resumes or aborts based on the result.

```
@app.post("/v1/approvals/{approval_id}/resolve")
def resolve(approval_id: str, body: ResolveIn, x_api_key: str = Header(...)):
    org = resolve_org(x_api_key) or _401()
    appr = db.update_approval(org.id, approval_id, body.status, body.comment, body.approver_id)
    record_event(... type="approval_action", payload={
        "approval_id": approval_id, "status": body.status,
        "approver_id": body.approver_id, "comment": body.comment})
    return {"approval_id": approval_id, "status": body.status}
```

WEEK 11 The thick-SDK pattern (so you're not a single point of failure)

WHY THIS MATTERS

A naive design calls your server before every action — adding a network round-trip to your customer’s critical path and making your uptime gate their production AI. Solution: ship a **signed policy bundle** to the SDK so the common Green/Yellow cases are decided *locally* in microseconds; only Orange/Red (a small fraction) call your server. Event recording is always asynchronous and buffered. This gives a strong latency story and means your downtime only affects the rare escalation path — which is also genuinely harder engineering than a dumb proxy, i.e. real deep-tech.

Day 49–52 — Local evaluation + buffered async recording + p99 budget

p99 = the latency your slowest 1% of calls experience (sort all calls slowest-to-fastest; the 99th-percentile one). Buyers specify SLAs at p99, not average, because averages hide tail pain. Target: local precheck p99 < 5ms; server escalation p99 < 150ms.

```
# SDK: decide locally, escalate only when needed, never block on recording
def precheck_local(self, action_type, content):
    features = extract_features_local(action_type, content)
    tier = self.policy_bundle.evaluate(action_type, features) # microseconds
    if tier in ("orange", "red"):
        return self._server_precheck(action_type, content) # rare network call
    self._buffer_event(...) # async flush
    return {"tier": tier, "decision": "allow"}
```

PHASE 3 DONE WHEN

A workflow runs through precheck; Green/Yellow decide locally and fast; a Red case blocks, lands in the dashboard, gets approved by a human, and resumes — with every decision and approval recorded as signed, anchored events. The enforcement layer is now demoable end-to-end.

Phase 4 (Week 12) — Pilot hardening, evidence bundle & the pitch

Day 53–55 — Migrate signing to a UAE-region KMS

Replace the file-based key with a cloud KMS/HSM (AWS KMS in me-central-1, or Azure Key Vault UAE North). The private key never leaves the hardware; signing is an API call that is itself logged. Swap `sign_hex()` to call the KMS. This is the difference between a demo and something a bank's security review will accept.

REGULATORY NOTE

In-region keys + in-region data directly answer PDPL data-residency expectations and the data-sovereignty concern that dominates MENA buying. Make “your prompts, content, and proofs never leave the UAE” a contractual default, not a config toggle — it's a differentiator hyperscaler guardrail features don't lead with.

Day 56–58 — Polish the evidence bundle & get a lawyer to sanity-check it

Finalise the exported bundle + standalone `verify.py` + a human-readable verification README. Spend a few billed hours with a UAE data-protection / financial-regulation lawyer to confirm the bundle and your erasure story would satisfy the UAE Data Office and CBUAE supervisory dialogue. High ROI — it makes your claims defensible.

Day 59–60 — The design-partner pitch

Land one named CBUAE-regulated design partner (even unpaid) for the logo and testimonial. Your pitch, in their language:

THE PITCH

“The CBUAE Guidance (Feb 2026) expects you to maintain a model inventory, document AI risk decisions, and provide human oversight of automated outcomes — with PDPL full-compliance due Jan 2027. Wathiq gives you all three as infrastructure: an independently-verifiable record of every AI action, human approval gates for high-risk cases, and an evidence bundle your regulator can verify without trusting either of us. In-region, model-agnostic, and built for UAE rules specifically.”

PROJECT DONE WHEN

Live demo: run a workflow → trigger a Red case → human approves in the dashboard → export the bundle → the risk officer runs your `verify.py` and sees `ALL EVENTS VERIFIED` on their own machine. That moment *is* the company.

Appendix A — Reference policy pack (CBUAE + PDPL mapped)

Starter rules to ship as defaults that institutions customise. Each maps to a named regulatory expectation so the “why” travels with the rule. This is a sales asset as much as a config file.

Rule	Condition	Tier	Maps to
Uncited factual claim	output asserts figures/facts with citation_count = 0	YELLOW	CBUAE transparency/explainability
PII in output	Emirates ID / IBAN / phone detected	ORANGE	PDPL data minimisation
Individualised financial advice	classifier: personalised buy/sell/allocation guidance	RED	CBUAE consumer protection; suitability
Cross-border data transfer	action would send personal data outside UAE w/o basis	RED	PDPL cross-border transfer rules
Discriminatory lending/pricing language	classifier: protected-attribute-linked adverse outcome	RED	CBUAE fairness/non-discrimination
Absolute/guaranteed-return claim	phrases like “guaranteed”, “risk-free”, “will return”	ORANGE	CBUAE consumer protection
Defamation/legal-risk language	classifier: unverified allegations about named parties	RED	General legal risk

Example JSON rules

```
[
  { "id": "fin_advice_red", "priority": 100,
    "match": { "action_type": "publish_report" },
    "condition": { "classifier": "individualised_financial_advice", "min_confidence": 0.7 },
    "decision": "require_approval", "tier": "red",
    "reason": "Possible individualised financial advice – CBUAE suitability/consumer protection." },

  { "id": "pii_orange", "priority": 90,
    "match": { "action_type": "*" },
    "condition": { "any_pii": true },
    "decision": "auto_mitigate", "tier": "orange",
    "reason": "Personal data detected – PDPL minimisation; redact before output." },

  { "id": "uncited_yellow", "priority": 50,
    "match": { "action_type": "generate_summary" },
    "condition": { "citation_count": 0 },
    "decision": "flag", "tier": "yellow",
    "reason": "Factual summary without citations – flag for reviewer." }
]
```

Heuristic risk score (v1 stub — calibrate later)

```
def score(f) -> float:
    s = 0.1 # baseline
    if f.get("any_pii"): s += 0.5
    if f.get("flagged_keyword"): s += 0.6
    if f.get("citation_count") == 0: s += 0.3
    return min(s, 1.0)
# Tiers (per-org configurable): <0.2 green | 0.2-0.5 yellow | 0.5-0.8 orange | >=0.8 red
```

REALITY CHECK

These weights are arbitrary placeholders with no validation. They are fine for a demo; they are **not** a defensible risk model. Be honest about this in writing. Real calibration needs labelled examples you don't yet have — collecting them from the pilot is itself a roadmap item.

Appendix B — Glossary for the newbie engineer

Term	Plain meaning
Hash (SHA-256)	A short fixed fingerprint of data. Change one character → totally different fingerprint. Can't be reversed.
Canonicalization	Writing data in one fixed form (sorted keys, no stray spaces) so identical meaning always yields the identical hash.
Signature (Ed25519)	Proof that the holder of a secret key vouched for some data. Private key signs; public key verifies.
Hash chain	Each record stores the previous record's hash, so altering an old record breaks every record after it.
Merkle tree / root	Combines many hashes into one root hash; lets you prove one item belongs to a batch with a tiny proof.
Anchoring	Committing a hash to a place you can't edit (timestamp authority, public log) so you can't secretly rewrite history.
RFC 3161 / TSA	A standard "trusted timestamp" service that signs "I saw this hash at this time."
KMS / HSM	Hardware/cloud vault that holds signing keys so they never leave it; every use is logged.
Crypto-shredding	"Deleting" data by destroying its encryption key, making it unrecoverable while keeping proof it existed.
Multi-tenancy	Many customers on one system, each fully isolated by an <code>org_id</code> filter on every query.
Fail-closed / fail-open	On error: block the action (closed, for high-risk) vs allow with a flag (open, to avoid downtime).
p50 / p95 / p99	Latency percentiles. p99 = your slowest 1% of requests. SLAs are quoted at the tail, not the average.
OPA / Rego	An industry-standard policy engine + language to migrate to when JSON rules outgrow their usefulness.

Appendix C — Honest limitations & what NOT to claim

Stating your boundaries plainly is what makes regulated buyers trust you. Put a version of this in your security documentation.

THE INTEGRATION BOUNDARY

Your guarantee starts at *ingestion*. You can prove an event was recorded faithfully and never altered — you cannot prove the customer's app sent you *everything* rather than dropping an embarrassing event before it reached you. Narrow this gap with SDK sequence numbers and expected-count checks; never claim to fully close it.

GARBAGE IN

Crypto proves integrity, not truth. You prove “this output was produced and unchanged,” not “this output is correct.” Don't conflate the two in marketing.

DETECTION IS IMPERFECT

Classifiers have false positives and negatives. That is precisely why Red = human approval. Never market autonomous legality judgments.

ERASURE VS VERIFIABILITY

A crypto-shredded record can prove existence and destruction, but its *content* can no longer be replayed and content-verified. This is the correct trade-off under PDPL, but state it explicitly.

NON-BINDING ≠ OPTIONAL

The CBUAE Guidance is non-binding but supervisory. Don't tell customers it's “the law.” Tell them it shapes supervisory dialogue and regulatory assessment — which is true and still compelling.

FINAL WORD

Wathiq wins by being the thing a UAE regulator can verify without trusting anyone — proof, not trust. Build the provenance core so solidly that the verification demo is undeniable, layer enforcement on top for the financial-sector mandate, and let one regulated logo open the door to the government prize. Ship the demo. Everything else follows from it.